REACT ROUTING – COMPLETE NOTES

1. What is Routing?

Routing is the process of navigating between different pages or components of an application using URLs.

Example:

- `/` → Home Page
- `/about` → About Page
- `/contact` → Contact Page
- `/login` → Login Page

Why do we need Routing?

1. To navigate between different pages.
2. To avoid full-page reloads.
3. To make navigation faster.
4. To create Single Page Applications (SPA).
5. To display different components based on the URL.

---

2. What is React Router?

React Router is a library used to manage navigation and routing in React applications.

It allows us to display different components based on the current URL without reloading the entire website.

Installation

Open the terminal in your React project and execute:

```
npm install react-router-dom
```

After installation, we can use React Router components and hooks.

---

3. What is BrowserRouter?

BrowserRouter is a component that enables routing in a React application.

It uses the browser's History API to manage navigation and URL changes.

Example:

```
import { BrowserRouter } from "react-router-dom";
```

```
function App() {
  return (
    <BrowserRouter>
      <h1>My React Application</h1>
    </BrowserRouter>
  );
}

export default App;
```

Important Points:

- BrowserRouter enables routing functionality.
- It usually wraps the entire application.
- It manages browser navigation and URL history.
- All routing components must be inside the router.

---

4. Important Components of React Router

React Router provides several components to manage navigation.

A. BrowserRouter

- Enables routing functionality.
- Manages URL changes.
- Usually wraps the application.

B. Routes

- Acts as a container for Route components.
- Checks the current URL.
- Displays the matching component.

C. Route

- Defines a particular URL and its corresponding component.
- Uses two important properties:

1. `path` - Defines the URL.
2. `element` - Defines which component should be displayed.

Example:

```
<Route path="/about" element={<About />} />
```

When the user visits `/about`, the About component will be displayed.

D. Link

Link is used to navigate from one route to another without reloading the entire page.

Example:

```
import { Link } from "react-router-dom";

<Link to="/about">About</Link>
```

Important:

- `to` defines the destination URL.
- Link is similar to the HTML anchor tag.
- It provides client-side navigation.

E. NavLink

NavLink is similar to Link, but it can identify whether its route is currently active.

It is mainly used for navigation menus.

Example:

```
import { NavLink } from "react-router-dom";

<NavLink to="/home">Home</NavLink>
```

We can apply CSS to the active link.

Example:

```
<NavLink
  to="/about"
  className={({ isActive }) =>
    isActive ? "active" : ""
  }
>
  About
</NavLink>
```

---

5. Basic Working of React Router

Consider the following structure:

BrowserRouter | Routes | |------ Route ("/") → Home | |------ Route ("/about") → About | |------ Route ("/contact") → Contact

Working:

1. User visits a URL.
2. React Router checks the URL.
3. It finds the matching Route.
4. The corresponding component is displayed.

---

6. Complete Example of React Routing

App.jsx

```
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function Contact() {
  return <h1>Contact Page</h1>;
}

function App() {
  return (
    <BrowserRouter>

      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link> |
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>

        <Route path="/" element={<Home />} />

        <Route path="/about" element={<About />} />

        <Route path="/contact" element={<Contact />} />

      </Routes>

    </BrowserRouter>
```

```
    );
}

export default App;
```

Explanation:

- BrowserRouter enables routing.
- Link provides navigation.
- Routes contains all route definitions.
- Route connects a URL with a component.
- Only the matching component is displayed.

---

7. What is useNavigate()?

useNavigate() is a React Router hook used to navigate from one route to another through JavaScript.

Unlike Link, useNavigate() is useful when navigation should happen after an event or condition.

Example:

```
import { useNavigate } from "react-router-dom";

function Login() {

  const navigate = useNavigate();

  function handleLogin() {
    navigate("/home");
  }

  return (
    <button onClick={handleLogin}>
      Login
    </button>
  );
}

export default Login;
```

Important Points:

- Used for programmatic navigation.
- Works inside React components.
- Commonly used after login or form submission.

Navigate Backward and Forward

```
navigate(-1); // Go to previous page

navigate(1);  // Go to next page
```

---

8. What is Dynamic Routing?

Dynamic routing means creating routes where some part of the URL can change dynamically.

Example:

```
/student/101
/student/102
/student/103
```

Instead of creating separate routes for every student, we can create one dynamic route.

Example:

```
<Route path="/student/:id" element={<Student />} />
```

Here:

- `student` is a fixed URL.
- `:id` is a dynamic parameter.
- The value of ID can change.

---

9. What is useParams()?

useParams() is a React Router hook used to access dynamic values from the URL.

Example:

Route:

```
<Route path="/student/:id" element={<Student />} />
```

Component:

```
import { useParams } from "react-router-dom";

function Student() {
```

```
  const { id } = useParams();

  return (
    <h1>Student ID: {id}</h1>
  );
}

export default Student;
```

If the URL is:

```
http://localhost:5173/student/101
```

Output:

```
Student ID: 101
```

Important:

- useParams() returns an object.
- It is used to access dynamic URL parameters.
- Parameter names must match the names defined in the route.

---

10. What is useLocation()?

useLocation() is a React Router hook used to get information about the current URL.

Example:

```
import { useLocation } from "react-router-dom";

function Page() {

  const location = useLocation();

  return (
    <h1>{location.pathname}</h1>
  );
}

export default Page;
```

If the URL is:

```
http://localhost:5173/about
```

Output:

```
/about
```

Important:

- It returns the current location object.
- `pathname` gives the current URL path.
- It can also provide search parameters, hash and navigation state.

---

11. What is a 404 Page?

A 404 page is displayed when the user visits a URL that does not match any defined route.

We can use `*` as a wildcard path.

Example:

```
<Routes>

  <Route path="/" element={<Home />} />

  <Route path="/about" element={<About />} />

  <Route
    path="*"
    element={<h1>404 - Page Not Found</h1>}
  />

</Routes>
```

Important:

- `*` matches unmatched routes.
- It is commonly used to display a custom error page.

---

12. Difference Between Link and NavLink

Link	NavLink
Used for navigation.	Used for navigation.

Link	NavLink
Does not automatically identify active routes.	Identifies active routes.
Suitable for normal links.	Suitable for navigation menus.
Does not provide automatic active styling.	Supports active styling.

---

13. Difference Between Link and useNavigate()

Link	useNavigate()
Used directly in JSX.	Used inside JavaScript logic.
Navigation happens when clicked.	Navigation can happen after an event or condition.
Used for normal navigation.	Used for programmatic navigation.
Component	Hook

---

14. Important React Router Components and Hooks

Name	Type	Purpose
BrowserRouter	Component	Enables routing
Routes	Component	Contains route definitions
Route	Component	Defines URL and component
Link	Component	Navigates between pages
NavLink	Component	Navigation with active styling
useNavigate()	Hook	Programmatic navigation
useParams()	Hook	Gets dynamic URL parameters
useLocation()	Hook	Gets current URL information

---

15. Important Points to Remember

1. React Router is used for navigation in React applications.
2. BrowserRouter enables routing.
3. Routes contains Route components.
4. Route connects a URL with a React component.
5. Link is used for normal navigation.
6. NavLink is used for navigation with active styling.
7. useNavigate() is used for programmatic navigation.
8. useParams() is used to get dynamic URL values.
9. useLocation() provides information about the current URL.
10. A wildcard route (*) handles unmatched URLs.

16. Practice Tasks

1. Create a React application with Home, About and Contact pages.
2. Create a navigation menu using Link.
3. Create the same navigation menu using NavLink and highlight the active page.
4. Create a Login page and navigate to the Dashboard after clicking the Login button using useNavigate().
5. Create a dynamic Student Details page using useParams().
6. Create a Product Details page with dynamic product IDs.
7. Display the current URL using useLocation().
8. Create a custom 404 Page Not Found component.

End of React Routing Notes.